

Get started

- Overview
- Quickstart
- API keys
- Libraries
- Pricing

Interactions API 🧑‍🔬

- Coding agent setup

Models

- All models
- Gemini 3 🧑‍🔬
- Nano Banana
- Veo
- Lyria 3 🧑‍🔬
- Lyria RealTime 🧑‍🔬
- Imagen
- Text-to-speech
- Embeddings
- Robotics 🧑‍🔬

Core capabilities

- Text
- Image >
- Video >
- Documents
- Speech and audio >
- Thinking >
- Structured outputs
- Function calling
- Long context

Agents

- Overview
- Deep Research Agent 🧑‍🔬

Tools

- Overview
- Google Search

Try the new high-speed, cost-effective [Veo 3.1 Lite](#) model for video generation at scale.

Home > Gemini API > Docs

Interactions API

On this page ▼

- Basic interactions
- Conversation
 - Stateful conversation
 - Stateless conversation
- Multimodal capabilities
- ...

★ We have updated our [Terms of Service](#).

★ **Using a coding agent?** Connect it to the **Gemini Docs MCP** and install the **gemini-interactions-api** skill to give your assistant direct access to the latest developer docs and best practices. [Set up your coding agent →](#)

The Interactions API ([Beta](#)) is a unified interface for interacting with Gemini models and agents. As an improved alternative to the [generateContent](#) API, it simplifies state management, tool orchestration, and long-running tasks. For comprehensive view of the API schema, see the [API Reference](#). During the Beta, features and schemas are subject to [breaking changes](#). To get started quickly, try the [Interactions API quickstart notebook](#).

- ✓ General use
- Function calling
- Deep Research agent

The following example shows how to call the Interactions API with a text prompt.

Python JavaScript REST

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Tell me a short joke about programming."
)

print(interaction.outputs[-1].text)
```

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models
Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Realtime

Resources
Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

★ **Note:** Interaction objects are saved by default (**store=true**) to enable state management features and background execution. See [Data Storage and Retention](#) for details on retention periods and how to delete stored data or opt out.

Conversation

You can build multi-turn conversations in two ways:

- Statefully by referencing a previous interaction
- Statelessly by providing the entire conversation history

Stateful conversation

To continue a conversation, pass the `id` from the previous interaction to the `previous_interaction_id` parameter. The API remembers the conversation history, so you only need to send the new input. For details on which fields are inherited and which must be re-specified, see [Server-side state management](#).

Python JavaScript REST

```
from google import genai

client = genai.Client()

# 1. First turn
interaction1 = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Hi, my name is Phil."
)
print(f"Model: {interaction1.outputs[-1].text}")

# 2. Second turn (passing previous_interaction_id)
interaction2 = client.interactions.create(
    model="gemini-3-flash-preview",
    input="What is my name?",
    previous_interaction_id=interaction1.id
)
print(f"Model: {interaction2.outputs[-1].text}")
```

Retrieve past stateful interactions

Using the interaction `id` to retrieve previous turns of the conversation.

Python JavaScript REST

```
previous_interaction = client.interactions.get("<YOUR INTERACTION ID>")
```

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models
Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Resources

Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio >
Google Cloud Platform >
Speech and audio >

Policies

Thinking >
Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

Stateless conversation

You can manage conversation history manually on the client side.

Python

JavaScript

REST

```
from google import genai

client = genai.Client()

conversation_history = [
    {
        "role": "user",
        "content": "What are the three largest cities in Spain?"
    }
]

interaction1 = client.interactions.create(
    model="gemini-3-flash-preview",
    input=conversation_history
)

print(f"Model: {interaction1.outputs[-1].text}")

conversation_history.append({"role": "model", "content": interaction1.outputs})
conversation_history.append({
    "role": "user",
    "content": "What is the most famous landmark in the second one?"
})

interaction2 = client.interactions.create(
    model="gemini-3-flash-preview",
    input=conversation_history
)

print(f"Model: {interaction2.outputs[-1].text}")
```

Multimodal capabilities

You can use the Interactions API for multimodal use cases such as image understanding or video generation.

Multimodal understanding

You can provide multimodal input as base64-encoded data inline, by using the Files API for larger files, or by passing a publicly accessible link in the uri field. The code samples that follow demonstrate the public URL method.

Image understanding

Docs API reference

Overview

Get started

- Batch API
- Overview
- Flex inference
- Quickstart
- Priority inference
- API keys
- Context caching
- Libraries

Guides

- Interactions API** 
- File input >
- Coding agent setup
- OpenAI compatibility

Media resolution

- Models**
- Token counting
- All models
- Prompt engineering
- Gemini 3 
- Logs and datasets >
- Nano Banana
- Safety >
- Veo
- Frameworks >
- Lyria 3 

Realtime

- Imagen
- Release notes
- Text-to-speech
- Deprecations
- Embeddings
- Rate limits
- Robotics 
- Billing info

Migrate to Gen AI SDK

- Core capabilities**
- API troubleshooting
- Text
- Partner and library
- Integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

- Thinking >
- Structured outputs
- Terms of service
- Function calling
- Available regions
- Long context
- Abuse monitoring

Feedback information

Agents

- Overview
- Deep Research Agent 

Tools

- Overview
- Google Search

Search

/



Language ▾

[Get API key](#)

[Cookbook](#)

[Community](#)

[Sign in](#)

[Python](#)

[JavaScript](#)

[REST](#)

```
from google import genai
client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input=[
        {"type": "text", "text": "What does this audio say?"},
        {
            "type": "audio",
            "uri": "YOUR_URL ✎",
            "mime_type": "audio/wav"
        }
    ]
)
print(interaction.outputs[-1].text)
```

Video understanding

[Python](#)

[JavaScript](#)

[REST](#)

```
from google import genai
client = genai.Client()

print("Analyzing video...")
interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input=[
        {"type": "text", "text": "What is happening in this video? Provide a time"},
        {
            "type": "video",
            "uri": "YOUR_URL ✎",
            "mime_type": "video/mp4"
        }
    ]
)

print(interaction.outputs[-1].text)
```

Document (PDF) understanding

[Python](#)

[JavaScript](#)

[REST](#)

```
from google import genai
client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
```

Get started

- Batch API
- Overview
- Flex inference
- Quickstart
- Priority inference
- API keys
- Context caching
- Libraries

Guides

- Interactions API** 
- File input >
- Coding agent setup
- OpenAI compatibility

Media resolution

- Models**
- Token counting
- All models
- Prompt engineering
- Gemini 3 
- Logs and datasets >
- Nano Banana
- Safety >
- Veo
- Frameworks >
- Lyria 3 

Real-time

- Resources** 
- Imagen
- Release notes
- Text-to-speech
- Deprecations
- Embeddings
- Rate limits
- Robotics 
- Billing info

Migrate to Gen AI SDK

- Core capabilities**
- API troubleshooting
- Text
- Partner and library
- Images

Google AI Studio

Google Cloud Platform

Speech and audio

Policies

- Thinking >
- Structured outputs
- Terms of service
- Function calling
- Available regions
- Long context
- Abuse monitoring

Feedback information

Agents

- Overview
- Deep Research Agent 

Tools

- Overview
- Google Search

Search

/



Language ▾

[Get API key](#)

[Cookbook](#)

[Community](#)

[Sign in](#)

Python

JavaScript

REST

```
import base64
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-pro-image-preview",
    input="Generate an image of a futuristic city.",
    response_modalities=["IMAGE"]
)

for output in interaction.outputs:
    if output.type == "image":
        print(f"Generated image with mime_type: {output.mime_type}")
        # Save the image
        with open("generated_city.png", "wb") as f:
            f.write(base64.b64decode(output.data))
```

Configure image output

You can customize generated images using `image_config` within `generation_config` to control the aspect ratio and resolution.

Parameter	Options	Description
<code>aspect_ratio</code>	1:1, 2:3, 3:2, 3:4, 4:3, 4:5, 5:4, 9:16, 16:9, 21:9	Controls the width-to-height ratio of the output image.
<code>image_size</code>	1k, 2k, 4k	Sets the output image resolution.

Python

JavaScript

REST

```
import base64
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-pro-image-preview",
    input="Generate an image of a futuristic city.",
    generation_config={
        "image_config": {
            "aspect_ratio": "9:16",
            "image_size": "2k"
        }
    }
)
```

Get started

- Batch API
- Overview
- Flex inference
- Quickstart
- Priority inference
- API keys
- Context caching
- Libraries

Guides

- Interactions API 
- File input >
- Coding agent setup
- OpenAI compatibility

Media resolution

- Models
- Token counting
- All models
- Prompt engineering
- Gemini 3 
- Logs and datasets >
- Nano Banana
- Safety >
- Veo
- Frameworks >
- Lyria 3 

Resources

- Imagen
- Release notes
- Text-to-speech
- Deprecations
- Embeddings
- Rate limits
- Robotics 
- Billing info

Migrate to Gen AI SDK

- Core capabilities
- API troubleshooting
- Text
- Partner and library
- Integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

- Thinking >
- Structured outputs
- Terms of service
- Function calling
- Available regions
- Long context
- Abuse monitoring

Feedback information

Agents

- Overview
- Deep Research Agent 

Tools

- Overview
- Google Search

```
# Set up the wave file to save the output:
def wave_file(filename, pcm, channels=1, rate=24000, sample_width=2):
    with wave.open(filename, "wb") as wf:
        wf.setnchannels(channels)
        wf.setsampwidth(sample_width)
        wf.setframerate(rate)
        wf.writeframes(pcm)

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-2.5-flash-preview-tts",
    input="Say the following: WOOHOO This is so much fun!.",
    response_modalities=["AUDIO"],
    generation_config={
        "speech_config": {
            "language": "en-us",
            "voice": "kore"
        }
    }
)

for output in interaction.outputs:
    if output.type == "audio":
        print(f"Generated audio with mime_type: {output.mime_type}")
        # Save the audio as wave file to the current directory.
        wave_file("generated_audio.wav", base64.b64decode(output.data))
```

Multi-speaker speech generation

Generate speech with multiple speakers by specifying speaker names in the prompt and matching them in the `speech_config`.

The prompt should include the speaker names:

```
TTS the following conversation between Alice and Bob:
Alice: Hi Bob, how are you doing today?
Bob: I'm doing great, thanks for asking! How about you?
Alice: Fantastic! I just learned about the Gemini API.
```

Then configure the `speech_config` with matching speakers:

```
"generation_config": {
    "speech_config": [
        {"voice": "Zephyr", "speaker": "Alice", "language": "en-US"},
        {"voice": "Puck", "speaker": "Bob", "language": "en-US"}
    ]
}
```

Overview

Get started

Batch API

Overview

Flex inference

Quickstart

Priority inference

API keys

Context caching

Libraries

Guides

Interactions API 

File input >

Coding agent setup

OpenAI compatibility

Media resolution

Models

Token counting

All models

Prompt engineering

Gemini 3 

Logs and datasets >

Nano Banana

Safety >

Veo

Frameworks >

Lyria 3 

Realtime 

Resources

Imagen

Release notes

Text-to-speech

Deprecations

Embeddings

Rate limits

Robotics 

Billing info

Migrate to Gen AI SDK

Core capabilities

API troubleshooting

Text

Partner and library

Integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

Thinking >

Structured outputs

Terms of service

Function calling

Available regions

Long context

Abuse monitoring

Feedback information

Agents

Overview

Deep Research Agent 

Tools

Overview

Google Search

```
for output in interaction.outputs:
    if output.type == "audio":
        print(f"Generated audio with mime_type: {output.mime_type}")
        with open("music.mp3", "wb") as f:
            f.write(base64.b64decode(output.data))
    elif output.type == "text":
        print(f"Lyrics: {output.text}")
```

For full-length songs (up to ~4 minutes), use the `lyria-3-pro-preview` model:

Python JavaScript REST

```
interaction = client.interactions.create(
    model="lyria-3-pro-preview",
    input="An epic cinematic orchestral piece about a journey home. "
        "Starts with a solo piano intro, builds through sweeping "
        "strings, and climaxes with a massive wall of sound.",
    response_modalities=["AUDIO", "TEXT"],
    generation_config={
        "response_mime_type": "audio/wav"
    }
)
```

Agentic capabilities

The Interactions API is designed for building and interacting with agents, and includes support for function calling, built-in tools, structured outputs, and the Model Context Protocol (MCP).

Agents

You can use specialized agents like `deep-research-pro-preview-12-2025` for complex tasks. To learn more about the Gemini Deep Research Agent, see the [Deep Research](#) guide.

 **Note:** The `background=true` parameter is required and only supported when using agents.

Python JavaScript REST

```
import time
from google import genai

client = genai.Client()

# 1. Start the Deep Research Agent
initial_interaction = client.interactions.create(
    input="Research the history of the Google TPUs with a focus on 2025 and 2026"
```

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models
Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Resources

Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio

Google Cloud Platform

Speech and audio >

Policies

Thinking >
Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

This section explains how to use function calling to define custom tools and how to use Google's built-in tools within the Interactions API.

Function calling

★ Format the **result** field in a **function_result** as an object (like `{"items": [...]}`), a string (like stringified JSON), or a **list** of content objects (like text or image objects). Do not pass a raw list of arbitrary objects directly to **result**. If your tool returns a generic list, wrap it in a dictionary before passing it to the model.

Python JavaScript REST

```
from google import genai

client = genai.Client()

# 1. Define the tool
def get_weather(location: str):
    """Gets the weather for a given location."""
    return f"The weather in {location} is sunny."

weather_tool = {
    "type": "function",
    "name": "get_weather",
    "description": "Gets the weather for a given location.",
    "parameters": {
        "type": "object",
        "properties": {
            "location": {"type": "string", "description": "The city and state, e."
        },
        "required": ["location"]
    }
}

# 2. Send the request with tools
interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="What is the weather in Paris?",
    tools=[weather_tool]
)

# 3. Handle the tool call
for output in interaction.outputs:
    if output.type == "function_call":
        print(f"Tool Call: {output.name}({output.arguments})")
        # Execute tool
        result = get_weather(**output.arguments)

        # Send result back
        interaction = client.interactions.create(
            model="gemini-3-flash-preview",
            previous_interaction_id=interaction.id,
            input=[{
```

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models
Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Realtime

Resources 
Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio

Google Cloud Platform

Speech and audio >

Policies

Thinking >
Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

```
"description": "Schedules a meeting with specified attendees at a given t
"parameters": {
  "type": "object",
  "properties": {
    "attendees": {"type": "array", "items": {"type": "string"}},
    "date": {"type": "string", "description": "Date of the meeting (e
    "time": {"type": "string", "description": "Time of the meeting (e
    "topic": {"type": "string", "description": "The subject of the me
  },
  "required": ["attendees", "date", "time", "topic"],
},
},
]

history = [{"role": "user", "content": [{"type": "text", "text": "Schedule a meeti

# 1. Model decides to call the function
interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input=history,
    tools=functions
)

# add model interaction back to history
history.append({"role": "model", "content": interaction.outputs})

for output in interaction.outputs:
    if output.type == "function_call":
        print(f"Function call: {output.name} with arguments {output.arguments}")

# 2. Execute the function and get a result
# In a real app, you would call your function here.
# call_result = schedule_meeting(**json.loads(output.arguments))
call_result = "Meeting scheduled successfully."

# 3. Send the result back to the model
history.append({"role": "user", "content": [{"type": "function_result", "

interaction2 = client.interactions.create(
    model="gemini-3-flash-preview",
    input=history,
)
print(f"Final response: {interaction2.outputs[-1].text}")
else:
    print(f"Output: {output}")
```

Multimodal function results

The `result` field in a `function_result` accepts either a plain string or an array of `TextContent` and `ImageContent` objects. This lets you return images, such as screenshots or charts, alongside text from your function calls, so the model can reason over the visual output.

[Python](#) [JavaScript](#) [REST](#)

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Realtime

Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

Thinking
Structured outputs
Terms of service

Function calling
Available regions

Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

```
tools=functions
)

for output in interaction.outputs:
    if output.type == "function_call":
        print(f"Function call: {output.name}({output.arguments})")

        # 2. Execute the function and load the image
        # Replace with actual function call, pseudo code for reading image from disk
        with open("screenshot.png", "rb") as f:
            base64_image = base64.b64encode(f.read()).decode("utf-8")

        # 3. Return a multimodal result (text + image)
        call_result = [
            {"type": "text", "text": "Screenshot captured successfully."},
            {"type": "image", "mime_type": "image/png", "data": base64_image}
        ]

        response = client.interactions.create(
            model="gemini-3-flash-preview",
            tools=functions,
            previous_interaction_id=interaction.id,
            input=[{
                "type": "function_result",
                "name": output.name,
                "call_id": output.id,
                "result": call_result
            }]
        )
        print(f"Response: {response.outputs[-1].text}")
```

Built-in tools

Gemini comes with built-in tools like [Grounding with Google Search](#), [Grounding with Google Image Search](#), [Grounding with Google Maps](#), [Code execution](#), [URL context](#), and [Computer Use](#)

Grounding with Google Search

Python

JavaScript

REST

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Who won the last Super Bowl?",
    tools=[{"type": "google_search"}]
)

# Find the text output (not the GoogleSearchResultContent)
text_output = next((o for o in interaction.outputs if o.type == "text"), None)
if text_output:
    print(text_output.text)
```

Get started

- Batch API
- Overview
- Flex inference
- Quickstart
- Priority inference
- API keys
- Context caching
- Libraries

Guides

- Interactions API** 
- File input >
- Coding agent setup
- OpenAI compatibility

Media resolution

- Models
- Token counting
- All models
- Prompt engineering
- Gemini 3 
- Logs and datasets >
- Nano Banana
- Safety >
- Veo
- Frameworks >
- Lyria 3 

Realtime

- Imagen
- Release notes
- Text-to-speech
- Deprecations
- Embeddings
- Rate limits
- Robotics 
- Billing info

Migrate to Gen AI SDK

- Core capabilities
- API troubleshooting
- Text
- Partner and library
- Integrations

Google AI Studio >

Google Cloud Platform >

- Speech and audio >

Policies >

- Structured outputs
- Terms of service
- Function calling
- Available regions
- Long context
- Abuse monitoring

Feedback information

Agents

- Overview
- Deep Research Agent 

Tools

- Overview
- Google Search

```
interaction = client.interactions.create(
    model="gemini-3.1-flash-image-preview",
    input="Search for an image of a vintage gold bitcoin coin.",
    tools=[{
        "type": "google_search",
        "search_types": ["web_search", "image_search"]
    }]
)
```

Mandatory display requirements

To comply with [Google Search Terms of Service](#), your UI must implement two distinct levels of attribution:

1. Google Search Attribution

You must display the "Check on Google" search suggestions provided in the `google_search_result` block.

- **Field:** `rendered_content` (HTML/CSS)
- **Action:** Render this chip as-is near the model's response.

2. Publisher Attribution

You must provide a link to the "containing page" (the landing page) for every image displayed.

- **Field:** `url` (Found within the `result` array)
- **Requirement:** You must provide a direct, single-click path from the image to its containing source webpage. Using intermediate image viewers or multi-click paths is not permitted.

Handling grounded response

The following snippet demonstrates how to handle the interleaved response blocks for both raw image data and mandatory attribution.

Python

JavaScript

```
for output in interaction.outputs:
    # 1. Handle raw multimodal image data
    if output.type == "image":
        print(f" Image received: {output.mime_type}")
        # 'data' contains base64-encoded image content
        display_image(output.data, output.mime_type)
    # 2. Handle mandatory Search and Publisher attribution
    elif output.type == "google_search_result":
        # Display Google Search Attribution
        if output.rendered_content:
            render_html_chips(output.rendered_content)
```

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models

Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Realtime

Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio >
Google Cloud Platform >
Speech and audio >

Policies >

Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

```
{
  "type": "google_search_result",
  "call_id": "search_002",
  "rendered_content": "<div class=\"search-suggestions\">...</div>", // Google

  "result": [
    {
      "url": "https://example.com/source-page", // Publisher Attribution
      "title": "Source Page Title"
    }
  ]
}
```

Grounding with Google Maps

Grounding with Google Maps allows models to use Google Maps data for visual context, map pins, and location-based discovery.

Python

JavaScript

REST

```
from google import genai
client = genai.Client()
interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="What's the best coffee shop near me?",
    tools=[{"type": "google_maps"}]
)
```

Service usage requirements

When presenting results from Grounding with Google Maps, you must follow the [Google Maps Terms of Service](#). You must inform your users of the following and meet these display requirements:

- **Inform the user:** Immediately follow the generated content with the associated Google Maps sources. The sources must be viewable within one user interaction.
- **Display links:** Generate a link preview for each source (including review snippets if present).
- **Attribute to "Google Maps":** Follow the [text attribution guidelines](#).
- **Display the source title.**
- **Link to the source** using the provided URL.
- **Attribution Guidelines:** Do not modify the text "Google Maps" (capitalization, wrapping). Prevent browser translation using `translate="no"`.

Handling the response

Get started

- Batch API
- Overview
- Flex inference
- Quickstart
- Priority inference
- API keys
- Context caching
- Libraries

Guides

- Interactions API 
- File input >
- Coding agent setup
- OpenAI compatibility

Media resolution

Models

- Token counting
- All models
- Prompt engineering
- Gemini 3 
- Logs and datasets >
- Nano Banana
- Safety >
- Veo
- Frameworks >
- Lyria 3 

Realtime

- Imagen
- Release notes
- Text-to-speech
- Deprecations
- Embeddings
- Rate limits
- Robotics 
- Billing info

Migrate to Gen AI SDK

- Core capabilities
- API troubleshooting
- Text
- Partner and library integrations

Google AI Studio >

Google Cloud Platform >

- Speech and audio >

Policies >

- Thinking >
- Structured outputs
- Terms of service
- Function calling
- Available regions
- Long context
- Abuse monitoring

Feedback information

Agents

- Overview
- Deep Research Agent 

Tools

- Overview
- Google Search

Expected output schema

Expect the following output schema when using Grounding with Google Maps.

The **Result Block** (type: "google_maps_result") contains the structured place data.

```
{
  "type": "google_maps_result",
  "call_id": "maps_001",
  "result": {
    "places": [
      {
        "place_id": "ChIJ...",
        "name": "Blue Bottle Coffee", // Google Maps Source
        "url": "https://maps.google.com/?cid=...", // Google Maps Link
        "review_snippets": [
          {
            "title": "Amazing single-origin selections",
            "url": "https://maps.google.com/...",
            "review_id": "def456"
          }
        ]
      }
    ]
  },
  "widget_context_token": "widgetcontent/..."
},
"signature": "..."
}
```

The **Text Block** (type: "text") contains the generated content with inline annotations.

```
{
  "type": "text",
  "text": "Blue Bottle Coffee (4.5★) on Mint Plaza was rated highly online...",
  "annotations": [
    {
      "type": "place_citation",
      "place_id": "ChIJ...",
      "name": "Blue Bottle Coffee", // Google Maps Source
      "url": "https://maps.google.com/?cid=...", // Google Maps Link
      "review_snippets": [
        {
          "title": "Amazing single-origin selections",
          "url": "https://maps.google.com/...",
          "review_id": "def456"
        }
      ]
    },
    "start_index": 0,
    "end_index": 42
  ]
}
```

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models

Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Realtime

Resources 
Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
integrations

Google AI Studio

Google Cloud Platform

Speech and audio >

Policies

Thinking >
Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

Search

/



Language ▾

[Get API key](#)

[Cookbook](#)

[Community](#)

[Sign in](#)

Python

JavaScript

REST

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Summarize the content of https://www.wikipedia.org/",
    tools=[{"type": "url_context"}]
)
```

Handling the response

The following snippet demonstrates how to handle the response by extracting the text and inline citations (type `url_citation`).

Python

JavaScript

```
for output in interaction.outputs:
    if output.type == "text":
        print(output.text)
        if output.annotations:
            print("\nSources:")
            for annotation in output.annotations:
                if annotation.get("type") == "url_citation":
                    print(f"- {annotation['title']}: {annotation['url']}")
```

Expected output schema

Expect the following output schema when using URL context.

The **Call Block** (type: `url_context_call`) contains the URL the model attempted to read.

```
{
  "type": "url_context_call",
  "id": "browse_001",
  "arguments": {
    "urls": ["https://www.wikipedia.org/"]
  },
  "signature": "EkYKIGY50T..."
}
```

The **Result Block** (type: `url_context_result`) contains the status of the retrieval.

```
{
  "type": "url_context_result",
```

Get started

- Batch API
- Overview
- Flex inference
- Quickstart
- Priority inference
- API keys
- Context caching
- Libraries

Guides

- Interactions API 
- File input >
- Coding agent setup
- OpenAI compatibility

Media resolution

Models

- Token counting
- All models
- Prompt engineering
- Gemini 3 
- Logs and datasets >
- Nano Banana
- Safety >
- Veo
- Frameworks >
- Lyria 3 

Realtime

- Imagen
- Release notes
- Text-to-speech
- Deprecations
- Embeddings
- Rate limits
- Robotics 
- Billing info

Migrate to Gen AI SDK

- Core capabilities
- API troubleshooting
- Text
- Partner and library integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

- Thinking
- Structured outputs
- Terms of service
- Function calling
- Available regions
- Long context
- Abuse monitoring

Feedback information

Agents

- Overview
- Deep Research Agent 

Tools

- Overview
- Google Search

```
]
}
```

Computer use

Python JavaScript REST

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-2.5-computer-use-preview-10-2025",
    input="Search for highly rated smart fridges with touchscreen, 2 doors, around",
    tools=[{
        "type": "computer_use",
        "environment": "browser",
        "excludedPredefinedFunctions": ["drag_and_drop"]
    }]
)

# The response will contain tool calls (actions) for the computer interface
# or text explaining the action
for output in interaction.outputs:
    print(output)
```

Handling Computer Use function results

Since Computer Use is a client-side tool loop, you must execute the action (e.g., opening a browser) and send the result back to the model. When sending the `function_result` for actions like `open_web_browser`, make sure to pass the URL response in the results list as shown below:

```
{
  "type": "function_result",
  "name": "open_web_browser",
  "call_id": "5q6h0zz70",
  "result": [
    {
      "type": "text",
      "text": "{\"url\": \"https://google.com\", \"safety_acknowledgement\":true}"
    },
    {
      "type": "image",
      "data": "iVBORw0KGgoAAAANSUUEUgAA...",
      "mime_type": "image/png"
    }
  ]
}
```

Overview

Get started

Batch API

Overview

Flex inference

Quickstart

Priority inference

API keys

Context caching

Libraries

Guides

Interactions API

File input

Coding agent setup

OpenAI compatibility

Media resolution

Models

Token counting

All models

Prompt engineering

Gemini 3

Logs and datasets

Nano Banana

Safety

Veo

Frameworks

Lyria 3

Realtime

Resources

Imagen

Release notes

Text-to-speech

Deprecations

Embeddings

Rate limits

Robotics

Billing info

Migrate to Gen AI SDK

Core capabilities

API troubleshooting

Text

Partner and library integrations

Google AI Studio

Google Cloud Platform

Speech and audio

Policies

Thinking

Structured outputs

Terms of service

Function calling

Available regions

Long context

Abuse monitoring

Feedback information

Agents

Overview

Deep Research Agent

Tools

Overview

Google Search

Python

JavaScript

```
for output in interaction.outputs:
    if output.type == "text":
        print(output.text)
        if output.annotations:
            print("\nSources:")
            for annotation in output.annotations:
                if annotation.get("type") == "file_citation":
                    print(f"- {annotation['file_name']} ({annotation['document_uri']})
                    print(f"  Snippet: {annotation['source']}")
```

Expected output schema

Expect the following output schema when using File search.

The **Call Block** (type: "file_search_call") contains the call metadata.

```
{
  "type": "file_search_call",
  "id": "filesearch_001",
  "signature": "EKYKIGY50T..."
}
```

The **Result Block** (type: "file_search_result") contains the result metadata.

```
{
  "type": "file_search_result",
  "call_id": "filesearch_001",
  "signature": "EKYKIGY50T..."
}
```

The **Text Block** contains the generated text and inline citations.

```
{
  "type": "text",
  "text": "The book 'I, Claudius' is a historical novel by Robert Graves...",
  "annotations": [
    {
      "type": "file_citation",
      "document_uri": "fileSearchStores/my-store-name/documents/abc",
      "file_name": "book_summaries.pdf",
      "source": "Claudius is the narrator of this historical novel...",
      "start_index": 0,
      "end_index": 60
    }
  ]
}
```

Get started

- Batch API
- Overview
- Flex inference
- Quickstart
- Priority inference
- API keys
- Context caching
- Libraries

Guides

- Interactions API 
- File input >
- Coding agent setup
- OpenAI compatibility

Media resolution

- Token counting
- All models
- Prompt engineering
- Gemini 3 
- Logs and datasets >
- Nano Banana
- Safety >
- Veo
- Frameworks >
- Lyria 3 

Resources

- Imagen
- Release notes
- Text-to-speech
- Deprecations
- Embeddings
- Rate limits
- Robotics 
- Billing info

Migrate to Gen AI SDK

- Core capabilities
- API troubleshooting
- Text
- Partner and library
- Integrations

Google AI Studio >

Google Cloud Platform >

- Speech and audio >

Policies >

- Structured outputs
- Terms of service

- Function calling
- Available regions

- Long context
- Abuse monitoring

Feedback information

Agents

- Overview
- Deep Research Agent 

Tools

- Overview
- Google Search

```
        },
        },
        "required": ["city"],
    },
}

tools = [
    {"type": "google_search"}, # Built-in tool
    get_weather                # Custom tool (callable)
]

# Turn 1: Initial request with both tools enabled
interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="What is the northernmost city in the United States? What's the weather",
    tools=tools
)

for output in interaction.outputs:
    if output.type == "function_call":
        print(f"Function call: {output.name} (ID: {output.id})")
        # Execute your custom function locally
        result = {"response": "Very cold. 22 degrees Fahrenheit."}
        # Turn 2: Provide the function result back to the model.
        # Passing `previous_interaction_id` automatically circulates the
        # built-in Google Search context (and thought signatures) from Turn 1
        interaction_2 = client.interactions.create(
            model="gemini-3-flash-preview",
            previous_interaction_id=interaction.id,
            tools=tools,
            input=[{
                "type": "function_result",
                "name": output.name,
                "call_id": output.id,
                "result": json.dumps(result)
            }]
        )

    for output in interaction_2.outputs:
        if output.type == "text":
            print(output.text)
```

Understanding tool context circulation

Gemini 3 and newer models support **tool context circulation** to maintain a reliable "memory" of server-side actions. When a built-in tool (like Google Search) is triggered, the API generates specific `toolCall` and `toolResponse` parts. These parts contain the precise context the model needs to reason about those results in the next turn.

- Stateful (Recommended):** If you use `previous_interaction_id`, the API manages this circulation for you automatically.
- Stateless:** If you manage history manually, you must include these blocks exactly as they were returned by the API in your input array.

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models
Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Realtime

Resources
Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio

Google Cloud Platform

Speech and audio >

Policies

Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

```
input="What is the weather like in New York today?",
tools=[mcp_server],
system_instruction=f"Today is {today}."
)

print(interaction.outputs[-1].text)
```

Important notes:

- Remote MCP only works with Streamable HTTP servers (SSE servers are not supported)
- Remote MCP does not work with Gemini 3 models (this is coming soon)
- MCP server names shouldn't include "-" character (use snake_case server names instead)

Structured output (JSON schema)

Enforce a specific JSON output by providing a JSON schema in the `response_format` parameter. This is useful for tasks like moderation, classification, or data extraction.

Python

JavaScript

REST

```
from google import genai
from pydantic import BaseModel, Field
from typing import Literal, Union
client = genai.Client()

class SpamDetails(BaseModel):
    reason: str = Field(description="The reason why the content is considered spam")
    spam_type: Literal["phishing", "scam", "unsolicited promotion", "other"]

class NotSpamDetails(BaseModel):
    summary: str = Field(description="A brief summary of the content.")
    is_safe: bool = Field(description="Whether the content is safe for all audiences")

class ModerationResult(BaseModel):
    decision: Union[SpamDetails, NotSpamDetails]

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Moderate the following content: 'Congratulations! You've won a free car!'",
    response_format=ModerationResult.model_json_schema(),
)

parsed_output = ModerationResult.model_validate_json(interaction.outputs[-1].text)
print(parsed_output)
```

Combining tools and structured output

Combine built-in tools with structured output to get a reliable JSON object based on information retrieved by a tool.

Get started

- Batch API
- Overview
- Flex inference
- Quickstart
- Priority inference
- API keys
- Context caching
- Libraries

Guides

- Interactions API**
- File input
- Coding agent setup
- OpenAI compatibility

Media resolution

- Models
- Token counting
- All models
- Prompt engineering
- Gemini 3
- Logs and datasets
- Nano Banana
- Safety
- Veo
- Frameworks
- Lyria 3

Realtime

- Imagen
- Release notes
- Text-to-speech
- Deprecations
- Embeddings
- Rate limits
- Robotics
- Billing info

Migrate to Gen AI SDK

- API troubleshooting
- Text
- Partner and library
- Integrations

Google AI Studio

Google Cloud Platform

- Speech and audio

Policies

- Structured outputs
- Terms of service
- Function calling
- Available regions
- Long context
- Abuse monitoring

Feedback information

Agents

- Overview
- Deep Research Agent

Tools

- Overview
- Google Search

Search

/



Language

Get API key

Cookbook

Community

Sign in

```
tools=[{"type": "url_context"}]

)

parsed_output = ModerationResult.model_validate_json(interaction.outputs[-1].text)
print(parsed_output)
```

Advanced features

There are also additional advance features that give you more flexibility in working with Interactions API.

Streaming

Receive responses incrementally as they are generated.

When `stream=true`, the final `interaction.complete` event does not contain the generated content in the `outputs` field. It only contains usage metadata and the final status. You must aggregate `content.delta` events client-side to reconstruct the full response or tool call arguments.

Python

JavaScript

REST

```
from google import genai

client = genai.Client()

stream = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Explain quantum entanglement in simple terms.",
    stream=True
)

for chunk in stream:
    if chunk.event_type == "content.delta":
        if chunk.delta.type == "text":
            print(chunk.delta.text, end="", flush=True)
        elif chunk.delta.type == "thought":
            print(chunk.delta.thought, end="", flush=True)
    elif chunk.event_type == "interaction.complete":
        print(f"\n\n--- Stream Finished ---")
        print(f"Total Tokens: {chunk.interaction.usage.total_tokens}")
```

Streaming event types

When streaming is enabled, the API returns Server-Sent Events (SSE). Each event has an `event_type` field indicating its purpose. Full list of event types is available in the [API reference](#).

Event Type

Description

Get started

- Batch API
- Overview
- Flex inference
- Quickstart
- Priority inference
- API keys
- Context caching
- Libraries

Guides

- Interactions API** 
- File input >
- Coding agent setup
- OpenAI compatibility

Media resolution

Models

- Token counting
- All models
- Prompt engineering
- Gemini 3 
- Logs and datasets >
- Nano Banana
- Safety >
- Veo
- Frameworks >
- Lyria 3 

Realtime

- Imagen
- Release notes
- Text-to-speech
- Deprecations
- Embeddings
- Rate limits
- Robotics 
- Billing info

Migrate to Gen AI SDK

Core capabilities

- API troubleshooting
- Text
- Partner and library integrations

Google AI Studio >

Google Cloud Platform >

- Speech and audio >

Policies >

- Structured outputs
- Terms of service
- Function calling
- Available regions
- Long context
- Abuse monitoring

Feedback information

Agents

- Overview
- Deep Research Agent 

Tools

- Overview
- Google Search

Reconstructing the Interaction object from streaming events

Unlike non-streaming responses, streaming responses do **not** contain an `outputs` array. You must reconstruct outputs by accumulating content from the `content.delta` events.

Python

JavaScript

```
from google import genai

client = genai.Client()

stream = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Write a haiku about Python programming.",
    stream=True
)

# Accumulate outputs by index
outputs = {}
usage = None

for chunk in stream:
    if chunk.event_type == "content.start":
        outputs[chunk.index] = {"type": chunk.content.type}

    elif chunk.event_type == "content.delta":
        output = outputs[chunk.index]
        if chunk.delta.type == "text":
            output["text"] = output.get("text", "") + chunk.delta.text
        elif chunk.delta.type == "thought_signature":
            output["signature"] = chunk.delta.signature
        elif chunk.delta.type == "thought_summary":
            output["summary"] = output.get("summary", "") + getattr(chunk.delta, "summary", "")

    elif chunk.event_type == "interaction.complete":
        usage = chunk.interaction.usage

# Final outputs list (sorted by index)
final_outputs = [outputs[i] for i in sorted(outputs.keys())]
print(f"\n\nOutputs: {final_outputs}")
```

Streaming tool calls

When using tools with streaming, the model generates function calls as a sequence of `content.delta` events on the stream. Unlike text, tool arguments are delivered as complete JSON objects within a single `content.delta` event. The `outputs` array is empty in the `interaction.complete` event during streaming, you must capture tool calls from deltas as shown below.

Python

JavaScript

REST

Overview

Get started

Batch API

Overview

Flex inference

Quickstart

Priority inference

API keys

Context caching

Libraries

Guides

Interactions API 

File input >

Coding agent setup

OpenAI compatibility

Media resolution

Models

Token counting

All models

Prompt engineering

Gemini 3 

Logs and datasets >

Nano Banana

Safety >

Veo

Frameworks >

Lyria 3 

Realtime 

Resources

Imagen

Release notes

Text-to-speech

Deprecations

Embeddings

Rate limits

Robotics 

Billing info

Migrate to Gen AI SDK

Core capabilities

API troubleshooting

Text

Partner and library integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

Thinking >

Structured outputs

Terms of service

Function calling

Available regions

Long context

Abuse monitoring

Feedback information

Agents

Overview

Deep Research Agent 

Tools

Overview

Google Search

```
)

# A map to capture tool calls by their ID as they arrive
function_calls = {}

for chunk in stream:
    if chunk.event_type == "content.delta":
        if chunk.delta.type == "text" and chunk.delta.text:
            print(chunk.delta.text, end="", flush=True)

            elif chunk.delta.type == "function_call":
                print(f"\nExecuting {chunk.delta.name} immediately...")
                # result = my_tools[chunk.delta.name](**chunk.delta.arguments)
                function_calls[chunk.delta.id] = chunk.delta

            elif chunk.event_type == "interaction.complete":
                print("\n\nAll tools executed. Stream finished.")
```

Configuration

Customize the model's behavior with `generation_config`.

PythonJavaScriptREST

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Tell me a story about a brave knight.",
    generation_config={
        "temperature": 0.7,
        "max_output_tokens": 500,
        "thinking_level": "low",
    }
)

print(interaction.outputs[-1].text)
```

Thinking

Gemini 2.5 and newer models use an internal reasoning process called "thinking" before generating a response. This helps the model produce better answers for complex tasks like math, coding, and multi-step reasoning.

Thinking level

The `thinking_level` parameter lets you control the model's reasoning depth:

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models
Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Realtime

Resources
Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

longer to reach a first token, but the output will be more carefully reasoned.

Thinking Models

Thinking summaries

The model's thinking is represented as **thought blocks** (`type: "thought"`) in the response outputs. You can control whether to receive human-readable summaries of the thinking process using the `thinking_summaries` parameter:

Value	Description
auto	(Default) Returns thought summaries when available.
none	Disables thought summaries.

Python

JavaScript

REST

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Solve this step by step: What is 15% of 240?",
    generation_config={
        "thinking_level": "high",
        "thinking_summaries": "auto"
    }
)

for output in interaction.outputs:
    if output.type == "thought":
        print(f"Thinking: {output.summary}")
    elif output.type == "text":
        print(f"Answer: {output.text}")
```

Every thought block contains a `signature` field (a cryptographic hash of the internal reasoning state) and an optional `summary` field (a human-readable summary of the model's reasoning). The `signature` is always present, but a thought block may contain only a signature with no summary in these cases:

- **Simple requests:** The model didn't reason enough to generate a summary
- **`thinking_summaries: "none"`** : Summaries are explicitly disabled

Your code should always handle thought blocks where the `summary` is empty or absent. When managing conversation history manually (stateless mode), you must include thought blocks with their signatures in subsequent requests to validate authenticity.

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models
Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Realtime

Resources 
Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio

Google Cloud Platform

Speech and audio >

Policies

Thinking >
Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

```
if output.type == "text":  
    print(output.text)
```

Working with Gemini Files API

Upload files to the Gemini [Files API](#) before using them.

Python

JavaScript

REST

```
from google import genai  
import time  
import requests  
client = genai.Client()  
  
# 1. Download the file  
url = "https://github.com/philschmid/gemini-samples/raw/refs/heads/main/assets/cats-and-dogs.jpg"  
response = requests.get(url)  
with open("cats-and-dogs.jpg", "wb") as f:  
    f.write(response.content)  
  
# 2. Upload to Gemini Files API  
file = client.files.upload(file="cats-and-dogs.jpg")  
  
# 3. Wait for processing  
while client.files.get(name=file.name).state != "ACTIVE":  
    time.sleep(2)  
  
# 4. Use in Interaction  
interaction = client.interactions.create(  
    model="gemini-3-flash-preview",  
    input=[  
        {  
            "type": "image",  
            "uri": file.uri,  
        },  
        {"type": "text", "text": "Describe what you see."}  
    ],  
)  
for output in interaction.outputs:  
    if output.type == "text":  
        print(output.text)
```

Flex and Priority inference tiers

You can use inference tiers with the Interactions API to optimize for different workload needs:

- [Flex](#) (`flex`) for cost optimization; 50% off standard pricing.
- [Priority](#) (`priority`) for latency optimization; the highest reliability service tier.

Python

Javascript

REST

Docs

API reference

Get started

Batch API
Overview
Flex inference
Quickstart
Priority inference
API keys
Context caching
Libraries

Guides

Interactions API 
File input >
Coding agent setup
OpenAI compatibility

Media resolution

Models
Token counting
All models
Prompt engineering
Gemini 3 
Logs and datasets >
Nano Banana
Safety >
Veo
Frameworks >
Lyria 3 

Realtime

Imagen
Release notes
Text-to-speech
Deprecations
Embeddings
Rate limits
Robotics 
Billing info

Migrate to Gen AI SDK

Core capabilities
API troubleshooting
Text
Partner and library
Integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

Structured outputs
Terms of service
Function calling
Available regions
Long context
Abuse monitoring

Feedback information

Agents

Overview
Deep Research Agent 

Tools

Overview
Google Search

Property	Type	Description
id	string	Unique identifier for the interaction.
model / agent	string	The model or agent used. Only one can be provided.
input	Content[]	The inputs provided.
outputs	Content[]	The model's responses.
tools	Tool[]	The tools used.
previous_interaction_id	string	ID of the previous interaction for context.
stream	boolean	Whether the interaction is streaming.
status	string	Status: completed , in_progress , requires_action , failed , etc.
background	boolean	Whether the interaction is in background mode.
store	boolean	Whether to store the interaction. Default: true . Set to false to opt out.
usage	Usage	Token usage of the interaction request.

Supported models & agents

Model Name	Type	Model ID
Gemini 3.1 Flash-Lite Preview	Model	gemini-3.1-flash-lite-preview
Gemini 3.1 Pro Preview	Model	gemini-3.1-pro-preview
Gemini 3 Flash Preview	Model	gemini-3-flash-preview
Gemini 2.5 Pro	Model	gemini-2.5-pro
Gemini 2.5 Flash	Model	gemini-2.5-flash
Gemini 2.5 Flash-lite	Model	gemini-2.5-flash-lite

Overview

Get started

Batch API

Overview

Flex inference

Quickstart

Priority inference

API keys

Context caching

Libraries

Guides

Interactions API 

File input >

Coding agent setup

OpenAI compatibility

Media resolution

Models

Token counting

All models

Prompt engineering

Gemini 3 

Logs and datasets >

Nano Banana

Safety >

Veo

Frameworks >

Lyria 3 

Realtime 

Resources

Imagen

Release notes

Text-to-speech

Deprecations

Embeddings

Rate limits

Robotics 

Billing info

Migrate to Gen AI SDK

Core capabilities

API troubleshooting

Text

Partner and library integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

Thinking >

Structured outputs

Terms of service

Function calling

Available regions

Long context

Abuse monitoring

Feedback information

Agents

Overview

Deep Research Agent 

Tools

Overview

Google Search

You can use the `id` of a completed interaction in a subsequent call using the `previous_interaction_id` parameter to continue the conversation. The server uses this ID to retrieve the conversation history, saving you from having to resend the entire chat history.

Only the conversation history (inputs and outputs) is preserved using `previous_interaction_id`. The other parameters are **interaction-scoped** and apply only to the specific interaction you are currently generating:

- `tools`
- `system_instruction`
- `generation_config` (including `thinking_level`, `temperature`, etc.)

This means you must re-specify these parameters in each new interaction if you want them to apply. This server-side state management is optional; you can also operate in stateless mode by sending the full conversation history in each request.

Data storage and retention

By default, all Interaction objects are stored (`store=true`) in order to simplify use of server-side state management features (with `previous_interaction_id`), background execution (using `background=true`) and observability purposes.

- Paid Tier:** Interactions are retained for **55 days**.
- Free Tier:** Interactions are retained for **1 day**.

If you don't want this, you can set `store=false` in your request. This control is separate from state management; you can opt out of storage for any interaction. However, note that `store=false` is incompatible with `background=true` and prevents using `previous_interaction_id` for subsequent turns.

You can delete stored interactions at any time using the delete method found in the [API Reference](#). You can only delete interactions if you know the interaction ID.

After the retention period expires, your data will be deleted automatically.

Interactions objects are processed according to the [terms](#).

Best practices

- Cache hit rate:** Using `previous_interaction_id` to continue conversations allows the system to more easily utilize implicit caching for the conversation history, which improves performance and reduces costs.
- Mixing interactions:** You have the flexibility to mix and match Agent and Model interactions within a conversation. For example, you can use a specialized agent, like the Deep Research agent, for initial data collection, and then use a standard Gemini model for follow-up tasks such as summarizing or reformatting, linking these steps with the `previous_interaction_id`.

Overview

Get started

Batch API

Overview

Flex inference

Quickstart

Priority inference

API keys

Context caching

Libraries

Guides

Interactions API 

File input >

Coding agent setup

OpenAI compatibility

Media resolution

Models

Token counting

All models

Prompt engineering

Gemini 3 

Logs and datasets >

Nano Banana

Safety >

Veo

Frameworks >

Lyria 3 

Realtime 

Resources

Imagen

Release notes

Text-to-speech

Deprecations

Embeddings

Rate limits

Robotics 

Billing info

Migrate to Gen AI SDK

Core capabilities

API troubleshooting

Text

Partner and library

Integrations

Google AI Studio >

Google Cloud Platform >

Speech and audio >

Policies >

Thinking >

Structured outputs

Terms of service

Function calling

Available regions

Long context

Abuse monitoring

Feedback information

Agents

Overview

Deep Research Agent 

Tools

Overview

Google Search

Google Maps

Code execution

URL context

Computer Use 

File Search

Combine Tools and Function calling 

Live API

Overview 

Get started >

The Interactions API is currently in an early beta stage. We are actively developing and refining the API capabilities, resource schemas, and SDK interfaces based on real-world usage and developer feedback.

As a result, **breaking changes may occur**. Updates may include changes to:

- Schemas for input and output.
- SDK method signatures and object structures.
- Specific feature behaviors.

For production workloads, you should continue to use the standard `generateContent` API. It remains the recommended path for stable deployments and will continue to be actively developed and maintained.

Feedback

Your feedback is critical to the development of the Interactions API. Share your thoughts, report bugs, or request features on our [Google AI Developer Community Forum](#).

What's next

- Try the [Interactions API quickstart notebook](#).
- Learn more about the [Gemini Deep Research Agent](#).

Was this helpful?



[Send feedback](#)

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](#), and code samples are licensed under the [Apache 2.0 License](#). For details, see the [Google Developers Site Policies](#). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-04-03 UTC.